

LAPORAN PROJECT PENGEMBANGAN APLIKASI  
"BORROWMINDER" SEBAGAI SISTEM INFORMASI MANAJEMEN  
PEMINJAMAN BARANG BERBASIS WEB  
PEMROGRAMAN WEB CR002



Dosen Pengampu: Jefry Sunupurwa Asri, S.Kom., M.Kom.

Disusun Oleh:

TEGHAR ANANDA RIZKI

PROGRAM STUDI TEKNIK INFORMATIKA & SISTEM  
INFORMASI  
FAKULTAS ILMU KOMPUTER  
UNIVERSITAS ESA UNGGUL  
TANGERANG  
2025

## KATA PENGANTAR

Puji dan syukur penulis panjatkan kehadirat Allah SWT, karena atas rahmat, hidayah, dan karunia-Nya, penulis dapat menyelesaikan laporan proyek akhir Pemrograman Web yang berjudul **"Pengembangan Aplikasi 'BorrowMinder' Sebagai Sistem Informasi Manajemen Peminjaman Barang Berbasis Web Menggunakan Laravel dan Docker"** ini dengan baik dan tepat waktu.

Laporan ini disusun guna memenuhi salah satu syarat penilaian mata kuliah Pemrograman Web pada program studi Informatika / Sistem Informasi di [Nama Universitas/Kampus Lo]. Dalam penyusunan proyek dan laporan ini, penulis menyadari bahwa keberhasilan yang dicapai tidak lepas dari bimbingan, dukungan, serta bantuan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis ingin menyampaikan terima kasih yang sebesar-besarnya kepada:

1. Jefry Sunupurwa Asri, S.Kom., M.Kom, selaku Dosen Mata Kuliah Pemrograman Web yang telah memberikan ilmu, arahan, bimbingan, dan masukan yang sangat berharga selama proses perkuliahan dan pengerjaan proyek ini.
2. Orang tua dan keluarga tercinta yang senantiasa memberikan doa, dukungan moral, maupun material yang tiada henti.
3. Rekan-rekan mahasiswa seperjuangan yang telah saling bertukar pikiran, memberikan motivasi, dan kerja sama yang baik dalam menyelesaikan proyek akhir ini.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan, baik dari segi penyusunan, tata bahasa, maupun kedalaman analisis sistem yang dibangun. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca demi perbaikan dan pengembangan sistem yang lebih baik di masa yang akan datang.

Akhir kata, semoga aplikasi BorrowMinder dan laporan proyek akhir ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi yang berguna bagi pembaca maupun perkembangan ilmu teknologi informasi.

## ABSTRAK

Pencatatan transaksi peminjaman inventaris yang masih dilakukan secara konvensional sering kali menimbulkan risiko kehilangan data, ketidakakuratan informasi ketersediaan barang, serta lemahnya akuntabilitas pengawasan. Penelitian proyek akhir ini bertujuan untuk merancang dan membangun **BorrowMinder**, sebuah sistem informasi manajemen peminjaman barang berbasis web yang dirancang untuk mendigitalisasi serta memusatkan seluruh alur pencatatan transaksi data. Aplikasi ini dikembangkan menggunakan framework **Laravel** dengan pola arsitektur *Model-View-Controller* (MVC) serta didukung oleh **MariaDB** sebagai sistem manajemen basis data relasional. Guna memastikan konsistensi lingkungan pengembangan (*development environment*) dan kemudahan replikasi sistem, seluruh infrastruktur aplikasi diisolasi menggunakan teknologi *containerization* **Docker** dan **Docker Compose**. Saat ini, pengembangan sistem berada pada tahapan analisis sistem dan implementasi fitur inti backend. Hasil pengujian menunjukkan bahwa fungsionalitas utama aplikasi—mulai dari modul formulir input peminjaman data, otomatisasi pengalihan jalur (*routing*), panel kontrol pelacakan *progress* admin, hingga manipulasi penghapusan data transaksi—telah berhasil diimplementasikan dan berjalan dengan baik. Proyek ini diharapkan dapat menjadi fondasi sistem inventaris digital yang aman, transparan, dan efisien untuk dikembangkan lebih lanjut ke tahap produksi skala luas.

**Kata Kunci:** BorrowMinder, Laravel, Docker, Sistem Peminjaman Barang, *Containerization*, MVC.

## ABSTRACT

*Conventional inventory loan recording often poses risks of data loss, inaccurate item availability information, and weak oversight accountability. This final project aims to design and develop **BorrowMinder**, a web-based loan management information system designed to digitalize and centralize the entire data transaction workflow. The application is developed using the **Laravel** framework with the *Model-View-Controller* (MVC) architecture pattern and is supported by **MariaDB** as the relational database management system. To ensure consistency in the development environment and ease of system replication, the entire application infrastructure is isolated using **Docker** and **Docker Compose** containerization technology. Currently, system development is at the system analysis and core backend feature implementation stage. The test results show that the main functionality of the application—ranging from the loan data input form module, automated rerouting (*routing*), admin progress tracking control panel, to transaction data deletion manipulation—has been successfully implemented and is functioning properly. This project is expected to serve as a foundation for a secure, transparent, and efficient digital inventory system for further development into a broad-scale production phase.*

**Keywords:** BorrowMinder, Laravel, Docker, Equipment Loan System, *Containerization*, MVC.

## Daftar Isi

|                                                               |    |
|---------------------------------------------------------------|----|
| LAPORAN PROJECT SISTEM PEMINJAMAN BARANG.....                 | 1  |
| KATA PENGANTAR.....                                           | 2  |
| ABSTRAK .....                                                 | 3  |
| ABSTRACT .....                                                | 3  |
| BAB I: PENDAHULUAN .....                                      | 5  |
| 1.1 Latar Belakang Masalah.....                               | 5  |
| 1.2 Rumusan Masalah .....                                     | 5  |
| 1.3 Batasan Masalah.....                                      | 6  |
| 1.4 Tujuan Penelitian.....                                    | 6  |
| 1.5 Manfaat Penelitian.....                                   | 6  |
| BAB II: ANALISIS DAN PERANCANGAN SISTEM.....                  | 7  |
| 2.1 Analisis Kebutuhan Sistem (Spesifikasi Teknologi).....    | 7  |
| 2.2 Perancangan Arsitektur Aplikasi (Konsep MVC).....         | 7  |
| 2.3 Perancangan Basis Data (Database Design).....             | 8  |
| 2.4 Perancangan Alur Sistem (Workflow Diagram) .....          | 9  |
| BAB III: IMPLEMENTASI SISTEM.....                             | 10 |
| 3.1 Struktur Arsitektur Kontainer (Docker Configuration)..... | 10 |
| 3.2 Implementasi Komponen Model (Eloquent ORM).....           | 10 |
| 3.3 Implementasi Mekanisme Jalur (Routing) .....              | 11 |
| 3.4 Implementasi Komponen Controller (Logika Bisnis).....     | 11 |
| 3.5 Implementasi Komponen Antarmuka (Blade Views) .....       | 12 |
| BAB IV: HASIL DAN PENGUJIAN .....                             | 13 |
| 4.1 Implementasi Antarmuka Sistem (Halaman Aplikasi).....     | 13 |
| 4.2 Skenario Pengujian Sistem (Black-Box Testing).....        | 13 |
| 4.3 Analisis Hasil Pengujian .....                            | 15 |
| BAB V: KESIMPULAN DAN SARAN.....                              | 15 |
| 5.1 Kesimpulan.....                                           | 15 |
| 5.2 Saran dan Pengembangan Lanjutan .....                     | 16 |

## BAB I: PENDAHULUAN

### 1.1 Latar Belakang Masalah

Perkembangan teknologi informasi yang masif di era transformasi digital menuntut setiap instansi, organisasi, maupun lembaga pendidikan untuk meninggalkan proses bisnis konvensional dan beralih ke sistem yang terotomatisasi. Salah satu aspek operasional yang sering kali luput dari modernisasi sistem adalah pengelolaan inventaris dan prosedur pencatatan peminjaman barang. Di lingkungan internal organisasi atau kampus, aktivitas peminjaman sarana dan prasarana penunjang kegiatan operasional terjadi hampir setiap hari dengan intensitas yang cukup tinggi.

Namun, kendala utama yang sering dihadapi adalah mayoritas pencatatan transaksi peminjaman barang masih dilakukan secara manual—baik menggunakan buku log fisik, lembaran kertas, maupun aplikasi *spreadsheet* terpisah seperti Microsoft Excel yang belum terintegrasi ke dalam basis data terpusat. Metode konvensional seperti ini memicu timbulnya berbagai risiko teknis dan non-teknis yang merugikan organisasi, antara lain:

1. **Rentan Terhadap Kehilangan dan Kerusakan Data:** Buku log fisik atau lembaran kertas sangat rahasia dan mudah hilang, terselip, atau rusak akibat kelalaian manusia (*human error*).
2. **Ketidakakuratan Informasi Stok Barang:** Pengguna tidak dapat mengetahui status ketersediaan barang secara *real-time* sebelum mendatangi lokasi logistik, yang menyebabkan inefisiensi waktu.
3. **Lemahnya Akuntabilitas Pengawasan:** Validitas data peminjam sulit diverifikasi secara cepat, sehingga menyulitkan proses pelacakan tanggung jawab apabila terjadi keterlambatan pengembalian, kerusakan, atau kehilangan barang.

Melihat urgensi permasalahan di atas, diperlukan sebuah solusi digital yang mampu mengintegrasikan seluruh alur peminjaman ke dalam satu platform yang terpusat. Oleh karena itu, proyek akhir ini merancang dan membangun sebuah aplikasi bernama **BorrowMinder**.

BorrowMinder dikembangkan menggunakan framework modern **Laravel** untuk menjamin struktur kode yang aman dan terorganisir dengan pola MVC (*Model-View-Controller*). Selain itu, untuk mengatasi masalah klasik perbedaan lingkungan server (*environment*) yang sering memicu error saat aplikasi dipindahkan antar-perangkat, proyek ini mengadopsi teknologi *containerization* **Docker**. Melalui Docker, aplikasi, web server Nginx, dan database MariaDB dikemas ke dalam kontainer yang terisolasi, sehingga sistem dijamin dapat berjalan dengan konsisten dan stabil di laptop mahasiswa maupun server pengujian dosen.

### 1.2 Rumusan Masalah

Berdasarkan latar belakang masalah yang telah dipaparkan, maka rumusan masalah dalam proyek akhir ini adalah:

1. Bagaimana merancang dan membangun sistem informasi manajemen peminjaman barang berbasis web menggunakan framework Laravel yang aman dan responsif?

2. Bagaimana memisahkan hak akses fungsionalitas antara halaman input data pengguna umum dengan panel kontrol pelacakan progress milik admin?
3. Bagaimana mengonfigurasi dan mengisolasi lingkungan pengembangan aplikasi menggunakan Docker agar proses deployment dan manajemen database MariaDB dapat berjalan secara konsisten?

### 1.3 Batasan Masalah

Agar pembahasan dalam laporan proyek akhir ini tidak meluas dan tetap fokus pada tujuan utama, maka penulis membatasi ruang lingkup penelitian sebagai berikut:

1. Pengembangan aplikasi difokuskan pada modul inti peminjaman (*Create*) dan manajemen penghapusan data transaksi (*Delete*).
2. Sistem manajemen basis data (*DBMS*) yang digunakan dibatasi pada MariaDB versi 10.11 yang berjalan di dalam kontainer Docker.
1. Hak akses admin pada fase ini ditekankan pada pengujian jalur (*routing*) backend dan fungsionalitas monitoring status progress proyek akhir.
2. Tampilan antarmuka (*user interface*) dibangun menggunakan utilitas Tailwind CSS tanpa melibatkan framework JavaScript tambahan (seperti Vue atau React).

### 1.4 Tujuan Penelitian

Adapun tujuan yang ingin dicapai melalui pelaksanaan proyek akhir ini adalah:

1. Menghasilkan aplikasi **BorrowMinder** sebagai sistem informasi manajemen peminjaman barang digital yang efektif untuk mereduksi penggunaan kertas (*paperless*).
2. Mengimplementasikan pola arsitektur MVC (Model-View-Controller) pada Laravel untuk mempermudah pemeliharaan komponen kode program jangka panjang.
3. Menerapkan arsitektur Docker Compose untuk menyederhanakan proses konfigurasi server lokal (Nginx, PHP, MariaDB) dalam satu instruksi terpadu.

### 1.5 Manfaat Penelitian

Penelitian dan pengembangan aplikasi BorrowMinder ini diharapkan dapat memberikan manfaat yang signifikan bagi beberapa pihak, antara lain:

- **Bagi Penulis:** Sebagai sarana untuk mengimplementasikan teori pemrograman web, memperdalam pemahaman mengenai manipulasi database (CRUD) di Laravel, serta mengasah kemampuan adaptasi terhadap teknologi DevOps dasar (Docker).
- **Bagi Organisasi / Pengguna:** Mempermudah proses administrasi peminjaman barang, meminimalisir risiko kehilangan arsip data, serta meningkatkan transparansi akuntabilitas pelacakan inventaris.
- **Bagi Pembaca / Mahasiswa Lain:** Dapat dijadikan sebagai referensi akademis atau studi literatur pendukung bagi mereka yang ingin membangun aplikasi web serupa menggunakan kombinasi teknologi Laravel dan Docker.

## BAB II: ANALISIS DAN PERANCANGAN SISTEM

### 2.1 Analisis Kebutuhan Sistem (Spesifikasi Teknologi)

Untuk menghasilkan aplikasi yang stabil, performan, dan mudah dikembangkan di masa mendatang, proyek **BorrowMinder** menggunakan kombinasi teknologi (*tech stack*) modern berbasis *open-source*. Pemisahan fungsionalitas server diisolasi sepenuhnya menggunakan teknologi kontainer. Berikut adalah rincian spesifikasi teknologi yang digunakan:

1. **Framework Laravel (PHP 8.x):** Laravel dipilih sebagai fondasi utama backend karena menyediakan ekosistem yang matang untuk pengembangan aplikasi web berskala cepat. Fitur-fitur seperti *Eloquent ORM*, routing yang rapi, proteksi CSRF bawaan, serta arsitektur *Model-View-Controller* (MVC) sangat membantu dalam memisahkan logika bisnis dengan visual antarmuka pengguna.
2. **Database MariaDB (v10.11):**

Sistem Manajemen Basis Data (DBMS) relasional ini dipilih karena performanya yang cepat, ringan, dan memiliki kompatibilitas penuh dengan MySQL. MariaDB bertugas mengelola seluruh data transaksional, mulai dari data progress proyek hingga records peminjaman barang.

3. **Web Server Nginx:**

Digunakan sebagai web server berkinerja tinggi untuk melayani *request* HTTP dari browser pengguna, yang kemudian diteruskan ke kontainer PHP-FPM (Laravel) untuk dieksekusi.

4. **Docker & Docker Compose:**

Teknologi *containerization* ini menjadi komponen krusial dalam proyek ini. Docker berfungsi mengemas aplikasi beserta pustaka pendukungnya ke dalam satu kesatuan kontainer. Dengan Docker Compose, konfigurasi multiservice (Nginx, PHP, dan MariaDB) dapat dijalankan secara bersamaan lewat satu file konfigurasi (*docker-compose.yml*), sehingga menjamin aplikasi akan berjalan 100% sama di perangkat mana pun tanpa kendala "*it works on my machine*".

5. **Tailwind CSS:**

Framework CSS berbasis *utility-first* yang digunakan untuk merancang antarmuka admin maupun formulir input agar terlihat modern, responsif, dan rapi tanpa perlu menulis kode CSS mentah dari awal.

### 2.2 Perancangan Arsitektur Aplikasi (Konsep MVC)

Aplikasi BorrowMinder dirancang dengan menerapkan pola arsitektur **MVC (Model-View-Controller)** yang memisahkan program menjadi tiga komponen utama:

- **Model:** Merepresentasikan struktur data pada database. Dalam proyek ini, terdapat model Project untuk memantau progress tugas akhir dan model Peminjaman yang berinteraksi langsung dengan tabel transaksi peminjaman.

- **View:** Bagian antarmuka yang dilihat oleh pengguna (ekstensi .blade.php). Bagian ini menggunakan *Blade Templating Engine* milik Laravel untuk menyajikan data dinamis dari database ke dalam bentuk HTML/CSS.
- **Controller:** Jembatan penghubung yang memuat logika bisnis utama aplikasi. Controller menerima input dari View (melalui *Request* HTTP), memprosesnya ke Model (Database), lalu mengembalikan respon dalam bentuk pengalihan halaman (*Redirect*) atau penampilan halaman baru.

## 2.3 Perancangan Basis Data (Database Design)

Sistem ini menggunakan dua tabel utama yang saling mendukung di dalam database `crkosongdua_db`. Rancangan struktur detail masing-masing tabel adalah sebagai berikut:

### 1. Tabel: *projects*

Tabel ini berfungsi untuk menyimpan data identitas dan status pelacakan progress dari proyek akhir yang dikelola melalui Dashboard Admin.

| Nama Field      | Tipe Data           | Atribut / Keterangan                                        |
|-----------------|---------------------|-------------------------------------------------------------|
| id              | BigInteger          | Primary Key, Auto Increment                                 |
| title           | String<br>(Varchar) | Menyimpan judul proyek (Contoh: "Sistem Peminjaman Barang") |
| progress_status | String<br>(Varchar) | Status tahapan (Contoh: "Analisis Sistem & Fitur")          |
| created_at      | Timestamp           | Waktu data pertama kali dibuat                              |
| updated_at      | Timestamp           | Waktu data terakhir kali diperbarui                         |

### 2. Tabel: *peminjamans*

Tabel ini digunakan khusus untuk mencatat histori atau riwayat transaksi peminjaman barang yang diinput oleh pengguna.

| Nama Field | Tipe Data  | Atribut / Keterangan        |
|------------|------------|-----------------------------|
| id         | BigInteger | Primary Key, Auto Increment |



| Nama Field    | Tipe Data           | Atribut / Keterangan                            |
|---------------|---------------------|-------------------------------------------------|
| nama_peminjam | String<br>(Varchar) | Nama lengkap orang yang meminjam barang         |
| nama_barang   | String<br>(Varchar) | Nama atau jenis barang inventaris yang dipinjam |
| tgl_pinjam    | Date                | Tanggal penyerahan/peminjaman barang            |
| status        | String<br>(Varchar) | Status kondisi peminjaman (Default: "Dipinjam") |
| created_at    | Timestamp           | Waktu otomatis pencatatan data masuk            |
| updated_at    | Timestamp           | Waktu otomatis pembaruan data transaksi         |

## 2.4 Perancangan Alur Sistem (Workflow Diagram)

Alur jalannya data transaksi pada sistem BorrowMinder dirancang sebagai berikut:

1. **Input Pengguna:** Pengguna mengakses halaman /peminjaman dan mengisi data nama, barang, serta tanggal pada formulir yang tersedia.
2. **Validasi & Pemrosesan:** Ketika tombol submit ditekan, data dikirim menggunakan metode HTTP POST ke controller. Controller akan memeriksa validitas data.
3. **Penyimpanan Data:** Jika valid, data langsung disimpan ke dalam tabel peminjamans di dalam kontainer database MariaDB.
4. **Respon Balik (Redirect):** Setelah data berhasil tersimpan, controller secara otomatis mengalihkan (*redirect*) halaman admin ke /admin/daftar-peminjaman untuk menampilkan data terbaru di dalam tabel laporan, lengkap dengan opsi penghapusan data.

## BAB III: IMPLEMENTASI SISTEM

### 3.1 Struktur Arsitektur Kontainer (Docker Configuration)

Aplikasi **BorrowMinder** dijalankan di atas infrastruktur kontainer yang terisolasi. Seluruh layanan web server, framework, dan basis data didefinisikan di dalam file `docker-compose.yml`. Sistem menjalankan tiga layanan (*services*) utama:

- `crkosongdua_nginx`: Menggunakan citra (*image*) Nginx sebagai web server utama yang menjembatani *request* HTTP eksternal ke dalam sistem.
- `crkosongdua_php`: Berisi lingkungan eksekusi PHP 8.x yang mengonfigurasi komponen pustaka (*extensions*) pendukung framework Laravel.
- `crkosongdua_db`: Menggunakan citra MariaDB versi 10.11 sebagai database server lokal dengan konfigurasi nama basis data `crkosongdua_db`.

### 3.2 Implementasi Komponen Model (Eloquent ORM)

Model pada Laravel merepresentasikan struktur tabel di dalam database. Untuk menangani transaksi peminjaman barang, diimplementasikan model `Peminjaman.php` yang terletak pada direktori `app/Models/`.

PHP

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Peminjaman extends Model
{
    use HasFactory;

    protected $table = 'peminjamans';

    protected $fillable = [
        'nama_peminjam',
        'nama_barang',
        'tgl_pinjam',
        'status',
    ];
}
```

**Penjelasan Kode:** Properti `$table` digunakan untuk mendeklarasikan secara eksplisit bahwa model ini terhubung ke tabel `peminjamans`. Properti `$fillable` bertindak sebagai proteksi keamanan *Mass Assignment*, menentukan kolom mana saja yang diizinkan untuk menerima manipulasi data secara massal dari formulir luar.

### 3.3 Implementasi Mekanisme Jalur (Routing)

Konfigurasi *routing* pada file `routes/web.php` menjadi pengatur lalu lintas *request* URL aplikasi. Jalur akses dikelompokkan secara terstruktur untuk fungsionalitas publik dan administrasi:

PHP

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\PeminjamanController;
use App\Http\Controllers\AdminProjectController;

// Jalur Input Form Peminjaman (User)
Route::get('/peminjaman', [PeminjamanController::class, 'create'])->name('peminjaman.create');
Route::post('/peminjaman', [PeminjamanController::class, 'store'])->name('peminjaman.store');

// Jalur Panel Kontrol & Administrasi (Admin)
Route::get('/admin/dashboard', [AdminProjectController::class, 'index'])->name('admin.dashboard');
Route::get('/admin/edit-status/{id}', [AdminProjectController::class, 'edit'])->name('admin.edit');
Route::post('/admin/update-status/{id}', [AdminProjectController::class, 'update'])->name('admin.update');

Route::get('/admin/daftar-peminjaman', [PeminjamanController::class, 'index'])->name('admin.peminjaman.index');
Route::delete('/admin/peminjaman/{id}', [PeminjamanController::class, 'destroy'])->name('admin.peminjaman.destroy');
```

### 3.4 Implementasi Komponen Controller (Logika Bisnis)

Logika utama pemrosesan data diatur di dalam `PeminjamanController.php` pada folder `app/Http/Controllers/`. Controller ini menangani penampilan daftar data, penyimpanan records baru, hingga fungsi eliminasi data:

PHP

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Peminjaman;

class PeminjamanController extends Controller
{
    // Menampilkan riwayat peminjaman di halaman admin
    public function index()
    {
        $peminjamans = Peminjaman::orderBy('created_at', 'desc')->get();
        return view('admin.daftar_peminjaman', compact('peminjamans'));
    }

    // Menampilkan form input peminjaman barang
    public function create()
    {
        return view('peminjaman');
    }
}
```

```
// Menyimpan data peminjaman baru ke database
public function store(Request $request)
{
    $request->validate([
        'nama_peminjam' => 'required|string|max:255',
        'nama_barang' => 'required|string|max:255',
        'tgl_pinjam' => 'required|date',
    ]);

    Peminjaman::create([
        'nama_peminjam' => $request->nama_peminjam,
        'nama_barang' => $request->nama_barang,
        'tgl_pinjam' => $request->tgl_pinjam,
        'status' => 'Dipinjam',
    ]);

    return redirect()->route('admin.peminjaman.index')
        ->with('success', 'Data peminjaman berhasil disimpan!');
}

// Menghapus data transaksi peminjaman
public function destroy($id)
{
    $peminjaman = Peminjaman::findOrFail($id);
    $peminjaman->delete();

    return redirect()->route('admin.peminjaman.index')
        ->with('success', 'Data peminjaman berhasil dihapus!');
}
}
```

### 3.5 Implementasi Komponen Antarmuka (Blade Views)

Komponen *View* dirancang interaktif menggunakan Tailwind CSS dan engine *Blade Templating*. Penyesuaian khusus dilakukan pada file tampilan guna menanamkan identitas identifikasi aplikasi secara statis:

#### 1. Form Pembaruan Progress (*resources/views/admin/edit.blade.php*)

Pada file form update status admin, identitas kode dimodifikasi pada bagian penayangan judul proyek agar mencerminkan penamaan resmi aplikasi **BorrowMinder**:

#### HTML

```
<div class="div-control-group">
    <span class="block text-xs font-semibold text-gray-500 uppercase mb-1">Nama Project</span>
    <span class="text-sm font-bold text-white">BorrowMinder - {{ $project->title }}</span>
</div>
```

#### 2. Tabel Utama Dashboard (*resources/views/admin/dashboard.blade.php*)

Sinkronisasi penamaan juga diterapkan pada elemen baris tabel dalam dashboard monitoring admin utama untuk menjaga konsistensi visual sistem:

#### HTML

```

<td class="p-4 font-semibold text-white">BorrowMinder - {{ $project->title }}</td>
<td class="p-4">
  <span class="px-2.5 py-1 text-xs font-semibold rounded bg-gray-900 text-emerald-400 border border-emerald-800">
    {{ $project->progress_status }}
  </span>
</td>

```

## BAB IV: HASIL DAN PENGUJIAN

### 4.1 Implementasi Antarmuka Sistem (Halaman Aplikasi)

Fase pengembangan aplikasi **BorrowMinder** saat ini berfokus pada penyediaan panel kontrol administrasi dan manajemen data operasional peminjaman barang. Berikut merupakan beberapa halaman utama sistem yang telah berhasil diimplementasikan:

#### 1. Halaman Dashboard Admin Monitoring

Halaman ini (/admin/dashboard) berfungsi sebagai pusat pemantauan berkala (*tracking progress*) proyek akhir. Pada halaman ini, nama sistem telah berhasil disinkronisasikan secara visual dengan identitas aplikasi, serta menampilkan status kemajuan pengerjaan sistem yang terintegrasi secara dinamis dengan database.

**[TEMPATKAN GAMBAR: Masukkan screenshot dashboard utama lo yang ada tabel Judul Project "BorrowMinder - Sistem Peminjaman Barang" dan Status Progress "Analisis Sistem & Fitur"]**

#### 2. Halaman Formulir Update Status Progress

Halaman ini (/admin/edit-status/{id}) digunakan khusus oleh admin untuk mengubah tahapan pengerjaan proyek akhir melalui elemen antarmuka berupa menu pilihan (*dropdown select option*). Kolom nama proyek dikunci secara bawaan (*read-only*), namun penulisan judul sistem telah berhasil diseragamkan lewat berkas tampilan.

**[TEMPATKAN GAMBAR: Masukkan screenshot form Update Status Progress lo yang ada pilihan status baru "Analisis Sistem & Fitur" dan judul "BorrowMinder - Sistem Peminjaman Barang"]**

#### 3. Halaman Laporan Manajemen Peminjaman Barang

Halaman ini (/admin/daftar-peminjaman) menampilkan struktur laporan tabular yang memuat seluruh records data transaksi peminjaman barang (Nama Peminjam, Nama Barang, dan Tanggal Peminjaman) yang diambil langsung dari kontainer basis data MariaDB, lengkap dengan tombol aksi manipulasi data.

### 4.2 Skenario Pengujian Sistem (Black-Box Testing)

Metode pengujian yang dilakukan pada proyek akhir ini menggunakan teknik **Black-Box Testing**. Pengujian ini berfokus pada fungsionalitas masukan (*input*) dan keluaran (*output*) sistem tanpa harus melihat struktur kode internal aplikasi secara mendalam.

Berikut adalah tabel matriks pengujian fungsional yang telah dilakukan pada sistem BorrowMinder:

| No | Fitur / Fungsi yang Diuji                          | Skenario Pengujian                                                                                              | Hasil yang Diharapkan                                                                                                                          | Status Pengujian       |
|----|----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| 1  | <b>Penyimpanan Data Peminjaman</b><br>(peminjaman) | Menginput data nama peminjam, nama barang, tanggal pinjam, lalu menekan tombol submit form.                     | Data berhasil divalidasi, tersimpan ke basis data MariaDB, dan sistem melakukan <i>redirect</i> otomatis ke halaman laporan admin tanpa error. | <b>BERHASIL (PASS)</b> |
| 2  | <b>Sinkronisasi Judul Sistem (Views)</b>           | Membuka halaman dashboard admin utama dan halaman ubah status progress proyek.                                  | Elemen teks pada antarmuka menampilkan penamaan sistem secara konsisten, yaitu:<br><i>"BorrowMinder - Sistem Peminjaman Barang"</i> .          | <b>BERHASIL (PASS)</b> |
| 3  | <b>Pembaruan Status Proyek Akhir</b>               | Memilih status baru <i>"Analisis Sistem &amp; Fitur"</i> pada menu dropdown lalu menekan tombol <i>"Simpan"</i> | Data status di database ter-update, muncul notifikasi sukses berwarna hijau, dan teks kolom status pada tabel utama ikut berubah               | <b>BERHASIL (PASS)</b> |

|   |                                            |                                                                                                              |                                                                                                                                    |                        |
|---|--------------------------------------------|--------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|------------------------|
|   |                                            | <i>Perubahan".</i>                                                                                           | secara dinamis.                                                                                                                    |                        |
| 4 | <b>Eliminasi Data Peminjaman (Destroy)</b> | Menekan tombol aksi " <i>Hapus</i> " (berwarna merah/hijau) pada baris record tabel daftar peminjaman admin. | Sistem mengeksekusi penghapusan data, record hilang dari tabel secara instan, dan memunculkan kembali pesan sukses pembaruan data. | <b>BERHASIL (PASS)</b> |

### 4.3 Analisis Hasil Pengujian

Berdasarkan seluruh rangkaian skenario pengujian fungsionalitas di atas, sistem **BorrowMinder** menunjukkan performa yang stabil dan bebas dari kendala fatal *routing (error HTTP 404 atau 500)*. Alur logika bisnis yang diatur di dalam komponen *Controller* telah berhasil mengeksekusi instruksi penyimpanan (*store*) dan pembersihan data (*delete*) pada basis data relasional MariaDB di dalam ekosistem Docker Compose secara tepat sasaran.

Notifikasi umpan balik (*flash session message*) berupa kotak informasi sukses di bagian atas tabel admin juga telah tampil secara interaktif, menandakan bahwa pengelolaan data operasional telah memenuhi standar kebutuhan fungsional sistem informasi manajemen.

## BAB V: KESIMPULAN DAN SARAN

### 5.1 Kesimpulan

Berdasarkan seluruh tahapan perancangan, implementasi kode program, hingga rangkaian pengujian fungsional yang telah dilakukan pada sistem informasi **BorrowMinder**, maka dapat diambil beberapa kesimpulan sebagai berikut:

1. **Keberhasilan Digitalisasi Transaksi:** Aplikasi **BorrowMinder** telah berhasil dibangun sebagai solusi digital berbasis web untuk menggantikan metode pencatatan peminjaman barang konvensional. Fitur CRUD inti (khususnya fungsi *Create* untuk menyimpan data transaksi dan fungsi *Delete* untuk eliminasi records data) telah berjalan dengan stabil dan lurus tanpa kendala teknis.
2. **Konsistensi Lingkungan Kerja (Docker):** Implementasi arsitektur multi-kontainer menggunakan Docker Compose (Nginx, PHP-FPM, dan MariaDB) terbukti mampu mengisolasi lingkungan pengembangan secara konsisten. Hal ini mempermudah

proses pemeliharaan serta replikasi sistem tanpa adanya dependensi software yang konflik pada sistem operasi lokal.

3. **Kemudahan Monitoring Administrasi:** Penerapan modul panel kontrol (Dashboard Admin) memberikan kemudahan bagi pengawas dalam melacak tahapan kemajuan proyek (*progress tracking*) secara dinamis, di mana status terbaru (saat ini berada pada fase "*Analisis Sistem & Fitur*") terintegrasi secara *real-time* ke basis data relasional.

## 5.2 Saran dan Pengembangan Lanjutan

Meskipun fungsi-fungsi inti backend aplikasi telah berhasil diimplementasikan dengan baik, sistem ini masih memerlukan beberapa peningkatan agar siap digunakan secara optimal pada skala produksi nyata. Beberapa saran untuk pengembangan lanjutan di antaranya adalah:

1. **Implementasi Sistem Autentikasi Keamanan (Auth):** Perlu ditambahkan modul enkripsi login dan registrasi (menggunakan pustaka bawaan seperti Laravel Breeze atau Fortify) guna membatasi dan mengamankan seluruh jalur URL administrasi (/admin/\*) agar tidak dapat diakses secara ilegal oleh pengguna umum.
2. **Penyempurnaan Modul Pengembalian Barang:** Mengembangkan fungsionalitas edit status transaksi (*Update*) agar admin dapat mengubah kondisi barang dari status "*Dipinjam*" menjadi "*Kembali*" lengkap dengan pencatatan tanggal pengembalian otomatis.
3. **Optimalisasi UI/UX Frontend:** Mempercantik visualisasi halaman depan (*landing page* atau halaman katalog inventaris) menggunakan komponen transisi Tailwind CSS agar antarmuka aplikasi menjadi lebih interaktif, responsif, dan ramah pengguna (*user-friendly*).





